

Inventory Dashboard Explanation

Overview

The Inventory Dashboard is a Livewire component that displays real-time inventory status by aggregating data from three interconnected database tables. This document explains how the dashboard calculates and displays inventory information.

Table Architecture

The inventory system uses three interconnected tables to track stock movements, costs, and current balances:

1. inventory Table

- **Purpose:** Current balance snapshot (optimized for fast dashboard queries)
- **Primary Key:** `lot_code` (string, not auto-increment)
- **Key Fields:**
 - `lot_code` - Unique identifier for each inventory lot
 - `item_code` - Item identifier
 - `category` - Inventory category (RAW, WIP, FG)
 - `qty_available` - Current available quantity
 - `warehouse` - Warehouse location
 - `unit_cost` - Cost per unit
 - `total_value` - Total inventory value
- **Used For:** Quick totals by category/warehouse, low stock alerts, fast dashboard queries

2. inventory_transactions Table

- **Purpose:** Immutable transaction ledger (complete audit trail)
- **Key Fields:**
 - `lot_code` - Links to inventory lot
 - `item_code` - Item identifier
 - `category` - Inventory category (RAW, WIP, FG)
 - `txn_type` - Transaction type (receipt, consume, produce, delivery)
 - `qty` - Transaction quantity (positive for receipts/productions, negative for consumes/deliveries)
 - `txn_date` - Transaction date
 - `related_doc_type` - Related document type (GRN, ProductionOrder, etc.)
 - `related_doc_id` - Related document ID
 - `unit_cost` - Cost per unit at transaction time

- `total_cost` - Total cost of transaction
- **Used For:** Historical tracking, calculating received/consumed quantities, audit trail

3. `inventory_layers` Table

- **Purpose:** FIFO/LIFO costing layers (tracks cost by receipt date)
- **Key Fields:**
 - `lot_code` - Links to inventory lot
 - `item_code` - Item identifier
 - `category` - Inventory category
 - `qty_available` - Remaining quantity in this layer
 - `unit_cost` - Cost per unit for this layer
 - `total_cost` - Total cost of remaining quantity
 - `receipt_date` - Date when this layer was received
 - `warehouse` - Warehouse location
 - `source_doc_type` - Source document type
 - `source_doc_id` - Source document ID
- **Used For:** Cost allocation when consuming inventory (oldest layers first for FIFO), accurate cost tracking

Dashboard Calculations

The dashboard performs several key calculations in the `render()` method:

1. Raw Materials Balance (`getBalanceRawMaterialsQuantity()`)

Formula:

`Balance = Total Received - Total Consumed`

Implementation: - **Total Received:** Sum of all `inventory_transactions` where: - `category = 'RAW'` AND - `txn_type = 'receipt'`

- **Total Consumed:** Sum of all `inventory_transactions` where:
 - `category = 'RAW'` AND
 - `txn_type = 'consume'`
 - Plus completed production quantities from `production_order_items`

Why not use `inventory` table directly? The dashboard calculates the actual available balance after consumption, which may differ from the inventory table if transactions are missing or incomplete. This ensures accuracy by using the transaction ledger as the source of truth.

Code Location: `app/Livewire/InventoryDashboard.php` lines 184-204

2. Total Raw Materials Received (getTotalRawMaterialsReceived())

Formula:

```
SELECT SUM(qty)
FROM inventory_transactions
WHERE category = 'RAW' AND txn_type = 'receipt'
```

Code Location: app/Livewire/InventoryDashboard.php lines 209-225

3. Total Raw Materials Consumed (getTotalRawMaterialsConsumed())

Formula:

```
-- Explicit consume transactions
SELECT SUM(qty)
FROM inventory_transactions
WHERE category = 'RAW' AND txn_type = 'consume'

-- Plus completed production quantities
SELECT SUM(completed_quantity)
FROM production_order_items
```

Code Location: app/Livewire/InventoryDashboard.php lines 234-292

4. Work in Progress (WIP) (getWorkInProgressQuantity())

Formula:

```
SELECT SUM(quantity - COALESCE(completed_quantity, 0)) as wip_quantity
FROM production_order_items
WHERE production_order.status IN ('pending', 'in_production', 'ready_for_production')
AND quantity > COALESCE(completed_quantity, 0)
```

Code Location: app/Livewire/InventoryDashboard.php lines 161-178

5. Inventory by Category (getInventoryByCategory())

Base Query:

```
SELECT category, SUM(qty_available) as total_qty
FROM inventory
GROUP BY category
```

Special Handling: - For RAW category, the dashboard replaces the direct sum with the calculated balance (received - consumed) to ensure accuracy

Code Location: app/Livewire/InventoryDashboard.php lines 84-116, 501-512

6. Inventory by Warehouse (getInventoryByWarehouse())

Base Query:

```
SELECT warehouse, SUM(qty_available) as total_qty
FROM inventory
GROUP BY warehouse
```

Special Handling: - Adjusts RAW quantities to match the calculated balance (received - consumed) for consistency

Code Location: app/Livewire/InventoryDashboard.php lines 118-149, 514-517

7. Low Stock Items (getLowStockItems())

Query:

```
SELECT lot_code, item_code, qty_available, uom
FROM inventory
WHERE qty_available < 10
ORDER BY qty_available
LIMIT 5
```

Code Location: app/Livewire/InventoryDashboard.php lines 151-159, 485-489

8. Available Raw Materials for Production

Query:

```
InventoryTransaction::where('category', 'RAW')
    ->where('txn_type', 'receipt')
    ->with(['grn.purchaseOrder.jobOrder'])
    ->orderBy('txn_date', 'desc')
    ->orderBy('created_at', 'desc')
    ->limit(10)
    ->get()
```

Code Location: app/Livewire/InventoryDashboard.php lines 492-498

Data Flow Example

Scenario: Receiving Raw Materials via GRN

When a GRN (Goods Receipt Note) is created:

1. **inventory_transactions:** Insert receipt transaction

```
[
    'txn_type' => 'receipt',
    'category' => 'RAW',
    'qty' => 100,
    'unit_cost' => 10.00,
    'total_cost' => 1000.00,
    'txn_date' => '2025-01-15'
]
```

2. **inventory_layers**: Create a new FIFO layer

```
[
    'lot_code' => 'RAW-001',
    'qty_available' => 100,
    'unit_cost' => 10.00,
    'total_cost' => 1000.00,
    'receipt_date' => '2025-01-15'
]
```

3. **inventory**: Create/update balance record

```
[
    'lot_code' => 'RAW-001',
    'qty_available' => 100,
    'total_value' => 1000.00,
    'unit_cost' => 10.00
]
```

Dashboard Result: Raw Materials = 100

Scenario: Consuming Raw Materials for Production

When production consumes raw materials:

1. **inventory_transactions**: Insert consume transaction

```
[
    'txn_type' => 'consume',
    'category' => 'RAW',
    'qty' => -50,
    'unit_cost' => 10.00,
    'total_cost' => -500.00
]
```

2. **inventory_layers**: Reduce oldest layer (FIFO)

```
// Before: qty_available = 100
// After:  qty_available = 50
// If fully consumed, layer is deleted
```

3. **inventory:** Update balance

```
// Before: qty_available = 100
// After:  qty_available = 50
```

Dashboard Result: Raw Materials = 50 (Balance: 100 received - 50 consumed)

Dashboard Display Components

The dashboard view (`resources/views/livewire/inventory-dashboard.blade.php`) displays:

1. Summary Cards

- **Raw Materials Card:** Shows balance, total received, and total consumed
- **Work in Progress Card:** Shows current WIP quantity
- **Finished Goods Card:** Shows FG inventory total

2. Inventory by Category

- Displays total quantities grouped by category (RAW, WIP, FG)
- RAW category uses calculated balance, not direct inventory table sum

3. Inventory by Warehouse

- Displays total quantities grouped by warehouse
- RAW quantities adjusted to match calculated balance

4. Low Stock Items

- Lists items with `qty_available < 10`
- Shows lot code, item code, quantity, and UOM
- Limited to 5 items for performance

5. Available Raw Materials for Production

- Lists recent receipt transactions available for production
 - Shows transaction details, job order information
 - Allows creating production orders directly from dashboard
-

Key Features

1. Caching Mechanism

The dashboard implements caching to avoid expensive recalculations:

- `cachedBalanceRawMaterials` - Caches raw materials balance
- `cachedTotalReceived` - Caches total received quantity
- `cachedTotalConsumed` - Caches total consumed quantity
- `cachedWIP` - Caches work in progress quantity
- `cachedInventoryByCategory` - Caches category breakdown

Code Location: `app/Livewire/InventoryDashboard.php` lines 29-34, 48-55

2. FIFO Costing

The `inventory_layers` table tracks costs by receipt date:

- When consuming inventory, oldest layers are consumed first (FIFO)
- Each layer maintains its own `unit_cost` and `total_cost`
- Layers are deleted when fully consumed
- Cost allocation is handled by `InventoryCostingService`

Code Location: `app/Services/InventoryCostingService.php`

3. Transaction Integrity

All three tables are updated within database transactions to ensure data consistency:

- When recording a transaction, all three tables are updated atomically
- If any update fails, all changes are rolled back

4. Special RAW Calculation

The dashboard uses transaction-based calculation for RAW materials instead of direct inventory table sum:

- More accurate: Uses transaction ledger as source of truth
- Handles missing transactions: Can estimate from WIP + FG if transactions are missing
- Consistent: Ensures balance matches actual received - consumed

Why Three Tables?

`inventory_transactions`

- **Purpose:** Complete audit trail
- **Benefits:** Immutable history, traceability, compliance
- **Use Case:** Historical reports, audit queries, transaction analysis

`inventory_layers`

- **Purpose:** Costing accuracy
- **Benefits:** FIFO/LIFO support, accurate cost allocation, aging analysis
- **Use Case:** Cost calculations, financial reporting, inventory valuation

inventory

- **Purpose:** Fast dashboard queries
 - **Benefits:** Quick aggregations, current balances, performance
 - **Use Case:** Dashboard displays, real-time status, low stock alerts
-

Performance Optimizations

1. **Direct DB Queries:** Uses `DB::table()` facade for simple aggregations instead of Eloquent models
 2. **Query Limits:** Limits results (e.g., 5 low stock items, 10 available materials)
 3. **Caching:** Caches expensive calculations within the same request
 4. **Selective Loading:** Only loads necessary fields and relationships
 5. **Eager Loading:** Uses `with()` to prevent N+1 query problems
-

Common Operations

Add Stock (Receipt)

1. Insert record in `inventory_transactions` (txn_type: 'receipt')
2. Insert record in `inventory_layers` (new FIFO layer)
3. Insert/update record in `inventory` (update qty_available)

Move Stock (Consume/Produce)

1. Insert record in `inventory_transactions` (txn_type: 'consume')
2. Insert record in `inventory_transactions` (txn_type: 'produce')
3. Reduce layers in `inventory_layers` (FIFO consumption)
4. Create new layers if producing
5. Update both lots in `inventory`

Deliver Stock (Delivery)

1. Insert record in `inventory_transactions` (txn_type: 'delivery')
 2. Reduce layers in `inventory_layers` (FIFO consumption)
 3. Update record in `inventory` (decrease qty_available)
-

Error Prevention

1. **Never delete from `inventory_transactions`:** Transactions are immutable
2. **Keep layers FIFO sorted:** Sort by `receipt_date` for accurate costing

3. **Update all three tables in a transaction:** Ensures data consistency
 4. **Validate qty_available >= 0:** Check before consuming inventory
 5. **Reconcile inventory against inventory_layers:** Ensure consistency
-

Related Files

- **Component:** app/Livewire/InventoryDashboard.php
 - **View:** resources/views/livewire/inventory-dashboard.blade.php
 - **Service:** app/Services/InventoryService.php
 - **Costing Service:** app/Services/InventoryCostingService.php
 - **Models:**
 - app/Models/Inventory.php
 - app/Models/InventoryTransaction.php
 - app/Models/InventoryLayer.php
 - **Migrations:**
 - database/migrations/2025_10_03_120849_create_inventory_table.php
 - database/migrations/2025_10_03_120900_create_inventory_transactions_table.php
 - database/migrations/2025_10_20_182310_add_costing_fields_to_inventory_tables.php
-

Summary

The Inventory Dashboard provides a comprehensive view of inventory status by:

1. **Aggregating** data from three specialized tables
2. **Calculating** accurate balances using transaction-based logic
3. **Displaying** real-time status with performance optimizations
4. **Supporting** FIFO/LIFO costing through inventory layers
5. **Maintaining** complete audit trail through transactions

The three-table architecture ensures accuracy, performance, and compliance while providing the flexibility needed for complex inventory management scenarios.